



Sharad Institute of Technology College of Engineering Yadav (An Autonomous Institute)

(Approved by AICTE, New Delhi, Recognized by Government of Maharashtra & Affiliated to Shivaji University, Kolhapur)

Department of Computer Science & Engineering

Experiment No: 16

Aim: Use Geopandas/Folium to create geographical visualization.

Objective:

- To understand the basics of spatial data and geospatial formats such as Shapefiles or GeoJSON.
- To use GeoPandas for loading, manipulating, and analyzing spatial datasets.
- To visualize spatial data through static maps using GeoPandas.
- To integrate Folium for creating interactive, web-based maps.

Theory:

1. Geopandas:

- Extension of Pandas for geospatial data manipulation.
- GeoPandas is an **open-source** Python library that makes working with geospatial data easier by extending pandas data structures to allow spatial operations on geometric types.
- It enables spatial queries, map plotting, and file I/O (via Pyogrio) for formats like Shapefiles and GeoJSON, relying on Shapely for geometry, Matplotlib for plotting, and PyProj for projections.
- Syntax: `import geopandas as gpd`

2. Geodata:

- Type of numeric data representing geographic locations of objects (e.g., country, state, city).
- Geodata, or geospatial data, is information tied to specific geographic locations, stored and analyzed within Geographic Information Systems (GIS).
- It combines location, geometry, and attribute data to model the physical world, utilized in industries like urban planning, logistics, and environmental management.
- Associated with specific locations or regions on Earth.

3. Folium:

- Open-source Python library used to create interactive maps.
- Integrates well with Geopandas and other libraries for visually appealing and interactive geographical visualizations without requiring advanced web development skills.
- Folium is a powerful Python library used to visualize geospatial data on interactive Leaflet maps.
 - It acts as a wrapper (*Folium allows you to write simple Python code to control Leaflet.js. It handles the "ugly" work of translating your Python data (like coordinates or Pandas DataFrames) into the JavaScript and HTML needed to render the map in a browser*) for the Leaflet.js library, allowing developers to create web-based maps, add markers, popups, and custom tile sets directly from Python, often used for data science visualization and dashboards.
- Syntax: `import folium`

Methods:

1. `read_file()`: Reads geographical files.

Syntax: `variable = gpd.read_file("file_path")`

```
import geopandas as gpd

# Basic usage
gdf = gpd.read_file("path/to/your/file.shp")
```

Key Parameters

- **filename** : (str, path, or URL) Path to the file or a URL.
- **layer** : (str) Name of the layer if the file contains multiple, such as in a GeoPackage.
- **bbox** : (tuple/GeoSeries/GeoDataFrame) Filters data to a specific spatial bounding box (minx, miny, maxx, maxy).
- **rows** : (int or slice) Load specific rows (e.g., `rows=10` for the first 10, or `rows=slice(0, 100)` for a range).
- **engine** : (str) The library used to read the file: `"pyogrio"` (default if installed) or `"fiona"`.  GeoPandas +4

2. `Map()`: Creates a map object that acts as a container for visual elements and data.

Syntax: `folium.Map(location=[latitude, longitude], zoom_start=10)`

```
import folium
m = folium.Map(location=[latitude, longitude], zoom_start=integer, tiles='style')
```

- **location** : A list of [lat, lon] coordinates to center the map.
- **zoom_start** : An integer defining the initial magnification (higher is closer).
- **tiles** : The map background style (e.g., 'OpenStreetMap', 'Stamen Terrain', Or 'CartoDB positron'). [GeeksforGeeks +4](#)

3. **GeoJson()**: Encodes geographic data structures such as points or lines.

Syntax: folium.GeoJson(geojson_data, name='geojson')

```
# Initialize map
m = folium.Map(location=[lat, lon], zoom_start=4)

# Basic GeoJson addition
folium.GeoJson(data, name="geojson_layer").add_to(m)
```

Key Parameters

- **data** : The GeoJSON data to be displayed. This can be a URL string, a local path, a dictionary, or an object with a `__geo_interface__` (like a GeoPandas GeoDataFrame).
- **style_function** : A function that takes a GeoJSON feature and returns a dictionary of Leaflet style options (e.g., `fillColor`, `color`, `weight`, `opacity`).
- **highlight_function** : A function that provides style changes when a user hovers over a feature.
- **tooltip** : Accepts a [folium.GeoJsonTooltip](#) object to display data fields on hover.
- **popup** : Accepts a `folium.GeoJsonPopup` object to display interactive information when a feature is clicked.
- **marker** : Allows customizing point features using markers like `folium.Circle` or `folium.Marker` instead of the default blue icon.
- **zoom_on_click** : A boolean that, if True, centers and zooms the map to the clicked feature. [GitHub Pages documentation +7](#)

4. **add_to()**: Adds map elements such as markers or layers to a map.

Syntax: object.add_to(map)

```
element.add_to(map_or_group)
```

Components

- `element` : The object you want to add, such as `folium.Marker()`, `folium.LayerControl()`, or `folium.GeoJson()`.
- `.add_to()` : The method that connects the element to the map.
- `map_or_group` : The parent object, typically a `folium.Map` object (e.g., `m`) or a `FeatureGroup`. [GitHub Pages documentation +3](#)

5. **GeoJsonTooltip()**: Displays a tooltip with information on the map.

Syntax: `folium.GeoJsonTooltip(fields, aliases=name)`

```
folium.GeoJsonTooltip(  
    fields=["property_name1", "property_name2"], # List of keys from GeoJSON  
    aliases=["Label1", "Label2"],               # Human-readable labels (optional)  
    localize=True,                               # Localize numbers/dates  
    labels=True,                                 # Show labels (aliases) or just  
    style="background-color: white; color: black;" # CSS styling (optional)  
) .add_to(geojson_object)
```

Conclusion: By combining GeoPandas for spatial data manipulation and Folium for interactive mapping, we can create powerful and insightful geographical visualizations.

